

3-SAT

Algorytmy zaawansowane

Klaudia Buczek, Klara Pluta, Gabriela Warchoř

1 Przedstawienie problemu

Sformułowanie problemu:

Zaprojektuj i zaimplementuj algorytm sprawdzający spełnialność formuły logicznej w postaci 3-CNF w czasie krótszym niż $O(2^n)$, gdzie n to liczba zmiennych.

Wprowadźmy podstawowe pojęcia:

- Literał – zmienna zdaniowa lub jej negacja
- Klauzula – alternatywa literałów

Definicja 1.1. CNF (koniunkcyjna postać normalna) Formuła jest w koniunkcyjnej postaci normalnej, jeśli jest ona koniunkcją klauzul.

Struktura 3-CNF jest przykładem struktury CNF, w której każda klauzula składa się dokładnie z 3 literałów. Problem 3-SAT polega na określeniu, czy istnieje takie przypisanie wartości logicznych zmiennym w strukturze 3-CNF, które czyni całą tę formułę prawdziwą. Jest to problem NP-zupełny, czyli możemy sprawdzić poprawność rozwiązania w czasie wielomianowym, ale nie znamy algorytmu, który w takim samym czasie potrafiłby to rozwiązanie znaleźć. Często, aby pokazać, że problem jest NP-zupełny, sprowadza się go do problemu 3-SAT.

2 Opis rozwiązania problemu

Standardowe podejście do problemu 3-SAT opiera się na algorytmie brute-force, który sprawdza wszystkie 2^n możliwe kombinacje przypisań. Poszukujemy metod, które pozwolą nam zredukować ten czas.

W tym celu wybrałyśmy algorytm oparty na drzewie decyzyjnym. Dzięki temu przeszukujemy rozwiązania z jednoczesnym odrzuceniem ścieżek, które nie prowadzą do spełnienia formuły. Daje nam to lepszą złożoność niż $O(2^n)$.

2.1 Opis algorytmu

Algorytm opiera się na stwierdzeniu: aby klauzula złożona z trzech elementów (np. $x \vee y \vee z$) była prawdziwa, przynajmniej jeden z nich musi być prawdą. Algorytm wybiera niespełnioną klauzulę i dzieli problem na trzy mniejsze podproblemy:

1. Zakładamy, że pierwszy literal jest prawdziwy.
2. Zakładamy, że pierwszy literal jest fałszywy, ale drugi musi być prawdziwy.
3. Zakładamy, że pierwszy i drugi są fałszywe, więc trzeci musi być prawdziwy.

Kluczowym elementem wydajności jest proces upraszczania formuły po każdym przypisaniu wartości zmiennej.

Jeśli przypisana wartość sprawia, że literal w klauzuli staje się prawdziwy, cała klauzula jest usuwana z dalszego poszukiwania rozwiązania w danej gałęzi. Jeśli przypisana wartość sprawia, że literal staje się fałszywy, jest on usuwany z klauzuli. Jeśli usuniemy ostatni literal z klauzuli, algorytm przerywa ścieżkę, ponieważ formuła w tym miejscu jest niespełnialna.

2.2 Pseudokod

Algorytm 1 Algorytm rozwiązujący 3-SAT

1: procedure 3SAT(F)	▷ F – formuła
2: if F jest pusta then return true	
3: end if	
4: if F zawiera klauzulę pustą then return false	
5: end if	
6: C – pierwsza niespełniona klauzula ($l_1 \vee l_2 \vee l_3$)	
7: if 3SAT($F[l_1 = 1]$) then return true	
8: end if	
9: if 3SAT($F[l_1 = 0, l_2 = 1]$) then return true	
10: end if	
11: if 3SAT($F[l_1 = 0, l_2 = 0, l_3 = 1]$) then return true	
12: end if	
13: return false	
14: end procedure	

3 Analiza poprawności i złożoności

3.1 Analiza poprawności

Poprawność wynika z przeszukiwania przestrzeni dopuszczalnych rozwiązań.

Algorytm w każdym kroku wybiera niespełnioną klauzulę i rozgałęzia się na trzy przypadki, które sumarycznie wyczerpują wszystkie sytuacje, w których ta klauzula jest prawdziwa. Dzięki temu proces przeszukiwania daje nam pewność, że jeśli istnieje jakiekolwiek przypisanie spełniające formułę, zostanie ono znalezione w jednej z gałęzi drzewa.

Jeśli w wyniku przypisań w danej gałęzi powstanie pusta klauzula (wszystkie literały są fałszywe), algorytm uznaje tę ścieżkę za błędną i wycofuje się do ostatniego punktu decyzyjnego.

3.2 Analiza złożoności

Złożoność obliczeniowa zaimplementowanego algorytmu wynika bezpośrednio z głębokości i struktury drzewa rekurencji. W przeciwieństwie do sprawdzenia wszystkich 2^n kombinacji, algorytm redukuje liczbę rozważanych przypadków dzięki podziałowi problemu na podstawie struktury klauzul 3-CNF.

Działanie algorytmu w najgorszym przypadku można opisać za pomocą równania rekurencyjnego, które odzwierciedla trzy ścieżki decyzyjne dla wybranej klauzuli:

$$T(n) = T(n-1) + T(n-2) + T(n-3)$$

- $T(n-1)$: koszt sprawdzenia gałęzi, w której pierwszy literal jest prawdziwy.
- $T(n-2)$: koszt sprawdzenia gałęzi, w której pierwszy literal jest fałszywy, a drugi prawdziwy.
- $T(n-3)$: koszt sprawdzenia gałęzi, w której dwa pierwsze literały są fałszywe, a trzeci prawdziwy.

W każdym węźle rekurencji algorytm wykonuje dodatkowo operacje upraszczania formuły, polegające na usuwaniu spełnionych klauzul oraz redukcji fałszywych literałów.

Rozwiązaniem równania charakterystycznego $x^3 - x^2 - x - 1 = 0$ jest $\alpha \approx 1,84$. Oznacza to, że w najgorszym wypadku złożoność naszego algorytmu wyniesie $O(1,84^n) < O(2^n)$.

4 Opis wejścia i wyjścia

4.1 Format pliku wejściowego

Aplikacja odczytuje dane z plików tekstowych (.txt) umieszczonych w bieżącym katalogu.

- **Pierwsza linia:** Zawiera dwie liczby całkowite rozdzielone spacją: n (liczba zmiennych) oraz m (liczba klauzul).
- **Kolejne m linie:** Każda linia reprezentuje jedną klauzulę i zawiera dokładnie trzy liczby całkowite rozdzielone spacją (literały).
 - Liczba dodatnia i oznacza zmienną x_i (np. 1 to x_1).
 - Liczba ujemna $-i$ oznacza negację zmiennej $\neg x_i$ (np. -3 to $\neg x_3$).
 - Zmienne są indeksowane od 1 do n .

Przykład pliku wejściowego instancja.txt:

```
4 3
1 -2 3
-1 4 -3
2 -4 1
```

Powyższy zapis odpowiada formule: $(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_4 \vee \neg x_3) \wedge (x_2 \vee \neg x_4 \vee x_1)$.

4.2 Format pliku wyjściowego

Wynik działania algorytmu jest zapisywany w pliku tekstowym, co pozwala na szybką weryfikację poprawności.

- **Status:** Informacja, czy udało się znaleźć rozwiązanie (ROZWIĄZYWALNE lub NIEROZWIĄZYWALNE).
- **Rozwiązanie:** W przypadku znalezienia rozwiązania wypisywany jest ciąg wartości logicznych dla kolejnych zmiennych $x_1, x_2, \dots, x_n$ (gdzie 1 oznacza prawdę, a 0 fałsz).

Przykład pliku wyjściowego wynik.txt:

Status: ROZWIĄZYWALNE

Rozwiązanie: 1 0 1 1

Oznacza to, że formuła jest spełniona dla: $x_1 = 1, x_2 = 0, x_3 = 1, x_4 = 1$.